CSE 586 Project 1 Final Report

Body Pose Forecasting with a Transformer

Tzuhsin Chiu

Abstract

This project predicts future human body poses from past observations. We use the AMASS dataset and encode pose parameters into VPoser 32-D latent vectors. We implement an autoregressive Transformer model (with block prediction) to forecast poses over long horizons. We evaluate using MPJPE in 3D after decoding predicted latents back to SMPL pose parameters. We compare against Zero-velocity and Constant-velocity baselines and analyze failure cases with worst-case visualizations.

1 Introduction

Human motion prediction is useful in animation, human-computer interaction, and robotics. Given a short motion history, the goal is to predict how the body will move in the future. This is difficult for long horizons because small pose errors can accumulate over many frames.

In this project, we forecast body pose sequences for 1–5 seconds ahead. We focus on continuous pose forecasting in VPoser latent space rather than discrete token prediction. Our main model is a Transformer with self-attention, which uses information from all previous frames to predict future latent vectors. To reduce the number of autoregressive steps, we use block prediction: instead of predicting one frame at a time, we predict a block of k frames and then move forward by blocks.

We evaluate quantitative accuracy with MPJPE and conduct error analysis using best/average/worst cases.

2 Dataset and Preprocessing

Data is taken from AMASS. Pose is represented using SMPL parameters, and we ignore global rotation and translation to simplify alignment. Only pose parameters are used for forecasting; body shape is fixed.

2.1 VPoser Encoding/Decoding

VPoser is used to encode pose parameters into a 32-D latent vector and decode latent vectors back to SMPL pose_body parameters. During evaluation, we decode predicted latent vectors with VPoser and then compute 3D joint positions with the SMPL body model.

2.2 Downsampling and Representation

AMASS sequences are originally recorded at 120 fps. We downsample to 30 fps by keeping every 4th frame. This reduces the number of prediction steps and helps reduce drift from long autoregressive chains.

We use a pose representation based on the 21 main joints (63 pose parameters). After VPoser encoding, each time step is represented as one continuous 32-D latent vector.

3 Methodology

3.1 PoseTransformer Model

Our model takes a latent sequence as input and outputs latent vectors for future time steps. The Transformer uses self-attention to model how body motion depends on earlier frames. We add positional encoding so the model can use frame order.

The output layer is a linear regression head that outputs a 32-D latent vector. We do not use softmax because the task is continuous regression rather than classification.

3.2 Block Prediction

Long-horizon autoregression can amplify prediction errors. If we predict one frame at a time for 5 seconds at 30 fps, that is 150 future frames. We use block prediction with $k = 30$ frames per block. This changes the autoregressive process from 150 steps into 5 block steps. We first predict a block of future latent vectors, then use the last predicted vector to start the next block.

3.3 Loss Function

We train with a combined regression loss on the predicted future block. Let $\hat{Z} \in \mathbb{R}^{B\times k\times 32}$ be the predicted block, where B is the batch size and k is the block size, and Z be the ground-truth block. We use two MAE terms:

- **Position loss:** $L_{\text{pos}} = \text{MAE}(\hat{Z}, Z)$
- **Velocity loss:** $L_{\text{vel}} = \text{MAE}(\Delta\hat{Z}, \Delta Z)$, where $\Delta Z_t = Z_t - Z_{t-1}$

The final loss is

$$L = L_{\text{pos}} + 1.0 \cdot L_{\text{vel}}.$$

This adds supervision on motion change, not only absolute position, and helps the model learn smoother and more realistic temporal dynamics.

4 Experiments and Evaluation

4.1 Baselines

We compare our Pose Transformer with two classic trajectory baselines:

- **Zero-velocity:** repeat the last observed pose for all future frames.
- **Constant-velocity:** compute velocity from the last two frames and linearly extrapolate forward.

4.2 Evaluation Metric: MPJPE

We use MPJPE (Mean Per Joint Position Error) as the main evaluation metric. MPJPE is the mean Euclidean distance between predicted and ground-truth 3D joint positions. All predictions are compared in 3D joint space after decoding and body-model forward pass. Lower MPJPE means better pose prediction.

For qualitative analysis, we visualize best, average, and worst cases at 5 seconds. This helps interpret whether errors correspond to realistic motion changes or severe failures. See Figures 2, 3, and 4 for qualitative examples.

4.3 Training and Inference Setup

We train the Transformer in VPoser latent space. For each training sample, the model observes a short history of latent vectors and predicts future latent vectors.

We downsample the original AMASS sequences from 120 fps to 30 fps. This makes the forecasting steps match realistic video frame rates and reduces drift from long per-frame autoregression.

For long-horizon prediction, we use block prediction. With $k = 30$ frames per block and 30 fps, predicting 5 seconds corresponds to 150 future frames. Instead of updating for all 150 frames, we predict about 5 blocks ($150 / 30 = 5$). During decoding, each predicted block latent is decoded by VPoser back to SMPL pose parameters. Then we compute 3D joint positions and MPJPE against the ground truth.

The training loss is a combined regression loss on latent blocks. We use a position MAE term and a velocity MAE term, with $L = L_{\text{pos}} + 1.0 \cdot L_{\text{vel}}$. We use separate train and test splits from AMASS, with an optional validation split for model selection.

For qualitative plots, we compute MPJPE for each sequence and select: the best case (lowest MPJPE), the average case (closest to the mean MPJPE), and the worst case (highest MPJPE). These cases are shown at 5 seconds to highlight stable vs. failed predictions.

4.4 Error Accumulation Rate

Figure-style analysis can be supported by a simple slope estimate. For the Pose Transformer, MPJPE increases from 118.23 mm at 1 s to 132.22 mm at 2 s (+13.99 mm). From 2 s to 5 s, MPJPE increases from 132.22 mm to 148.32 mm (+16.10 mm over 3 seconds). Overall, the error increases by about 30.09 mm from 1 s to 5 s.

Compared to the baselines, the growth is slower for the Transformer. Zero-velocity increases from 196.27 mm at 1 s to 227.45 mm at 2 s (+31.18 mm) and to 266.05 mm at 5 s (+38.60 mm from 2 s to 5 s). Constant-velocity grows even faster: 348.44 mm at 1 s to 431.77 mm at 2 s (+83.33 mm) and to 483.73 mm at 5 s (+51.96 mm from 2 s to 5 s).

This confirms the expected behavior of autoregressive forecasting: long horizons lead to larger errors, but the Transformer reduces error growth rate. The shape of Figure 1 is therefore expected: all curves increase with time because autoregressive errors accumulate. Constant-velocity rises the fastest since real human motion is not strictly linear, while Zero-velocity is only competitive at very short horizon. The Transformer curve is flatter at medium and long horizons, which indicates better temporal motion modeling. Block prediction also helps by reducing the number of autoregressive updates, which slows down long-term error growth.

4.5 Visualization of Results

To make the trend easier to interpret, we provide both quantitative curves and qualitative examples. Figure 1 shows MPJPE vs. horizon for all three methods. Figures 2, 3, and 4 show worst/average/best qualitative cases at 5 seconds.

Time horizon	Zero-velocity (mm)	Constant-velocity (mm)	Pose Transformer (mm)
80 ms	37.2339	18.3643	30.44
320 ms	136.8174	155.1827	83.03
1 s	196.2699	348.4447	118.23
2 s	227.4510	431.7711	132.22
5 s	266.0536	483.7315	148.32

Table 1: MPJPE (mm) at different forecasting horizons. Transformer numbers are from `trans_results_150`.

5 Discussion: Error Analysis

The worst-case example is the most useful for understanding model limitations. At 5 seconds (pred_len=150), the notebook prints the following errors (mm) for visual cases: **best:** 43.97, **average:** 129.43, **worst:** 261.80.

5.1 Mean Regression

In long horizons, the prediction can become more conservative. When uncertainty increases, the model may regress toward an "average" latent pose. This can cause visible artifacts such as sudden changes in limb position. In the worst-case visualization, this appears as an unexpected arm drop, which matches the idea that the model is returning toward a safer mean motion. This behavior becomes more noticeable after about 2 seconds, where long-term uncertainty is higher.

5.2 Physical Inconsistency

Pose forecasting in latent space does not directly enforce physics or biomechanics constraints. Over long horizons, repeated prediction errors can accumulate. As a result, motion can look stiff, drift, or become physically inconsistent compared to human motion. In the worst case, two concrete artifacts are visible: (1) upper-limb posture collapses too early (arm drop), and (2) body motion becomes overly rigid with reduced natural swing, which looks less human-like.

5.3 How Block Prediction Helps

Block prediction reduces the number of autoregressive steps. Instead of running 150 step updates for 5 seconds, we run about 5 block updates with $k = 30$. This reduces error accumulation speed compared to pure frame-by-frame autoregression.

5.4 When Transformer Beats the Baselines

From Table 1, the Pose Transformer has lower MPJPE than both baselines at 1 s, 2 s, and 5 s. For example, at 1 s it is 118.23 mm vs. 196.27 mm (zero-velocity) and 348.44 mm (constant-velocity). At 5 s it is 148.32 mm vs. 266.05 mm (zero-velocity) and 483.73 mm (constant-velocity). At 80 ms, Constant-velocity is slightly better, but longer horizons show clear gains from the Transformer.

6 Limitations

Even if the Transformer has lower MPJPE than the baselines, there are still limitations. First, we predict in VPoser latent space and then decode back to SMPL. If the predicted latent vectors are slightly off, decoding can produce visible joint position errors. This is why the error curve usually increases with horizon.

Our evaluation also uses fixed body shape and removes global rotation and translation. That makes the MPJPE comparison easier, but it also means our results assume aligned bodies. If the input had different global poses or body sizes, additional normalization would be needed.

Second, the model does not explicitly enforce physical constraints. Human motion has many implicit rules (gravity, joint limits, body balance). Without constraints, the model may generate motion that is plausible in latent space but not physically stable after decoding. In the worst-case examples, we see motion that can look stiff or that changes abruptly.

Another limitation is the training objective. We use MAE between predicted and true latent vectors. The latent-space distance may not perfectly match the final joint error after decoding, so the model can focus on the wrong parts of the motion.

Third, the autoregressive strategy still matters. Block prediction reduces the number of steps, but prediction still depends on previous predictions. If the model makes an early mistake inside a block, the error can carry to later frames.

Potential improvements are:

- Use overlapping blocks or a smaller block size to reduce hard jumps between blocks.
- Add simple regularization to encourage smooth latent trajectories.
- Improve evaluation by reporting MPJPE curves for multiple horizons and adding subject-wise breakdown.
- Add a light smoothness penalty on predicted latent differences (to reduce sharp changes).

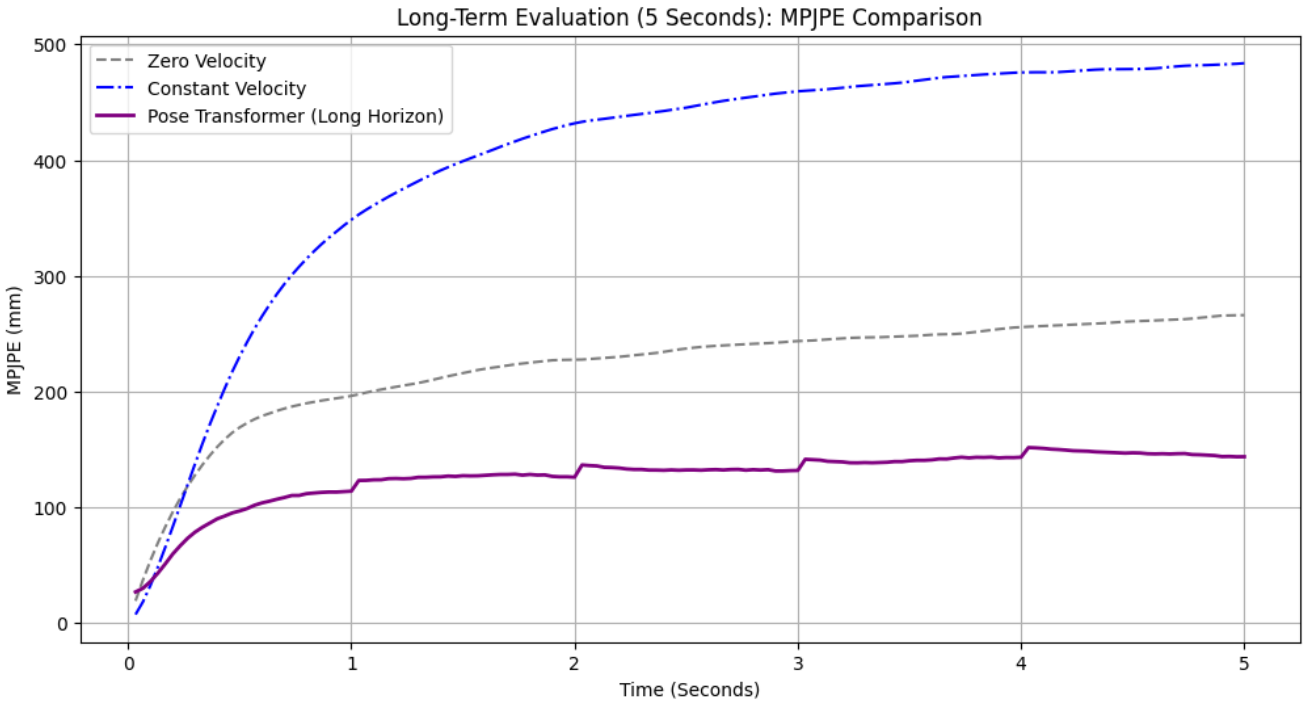

Figure 1: MPJPE vs. time horizon.

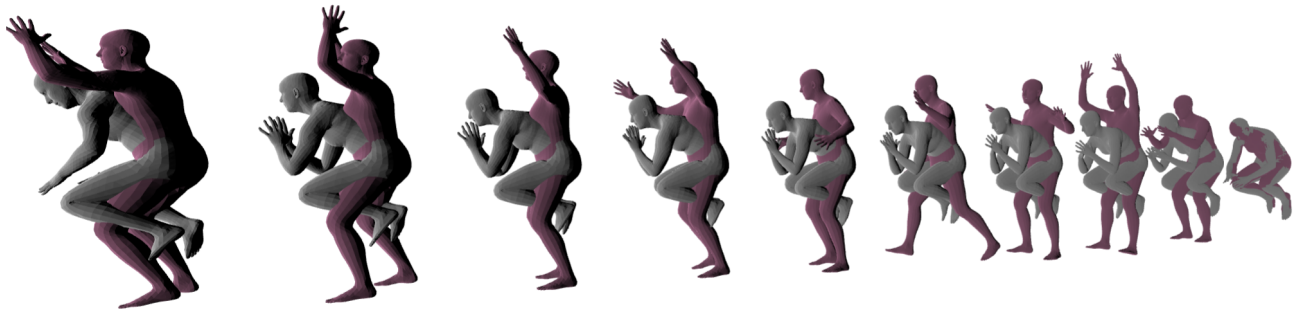

Figure 2: Worst-case visualization at 5 seconds.

- Explore decoding-time constraints or post-processing to reduce physically impossible poses.

7 Conclusion

We presented a Transformer-based pose forecasting model in VPoser latent space. We downsample AMASS to 30 fps and use block prediction to improve long-horizon stability. Quantitative results show that the model can predict future motion and reach 5-second forecasts, although error still grows with horizon.

Overall, this project shows that attention-based sequence models can learn long-range pose structure, but further work is needed to improve stability and physical consistency at extreme horizons.

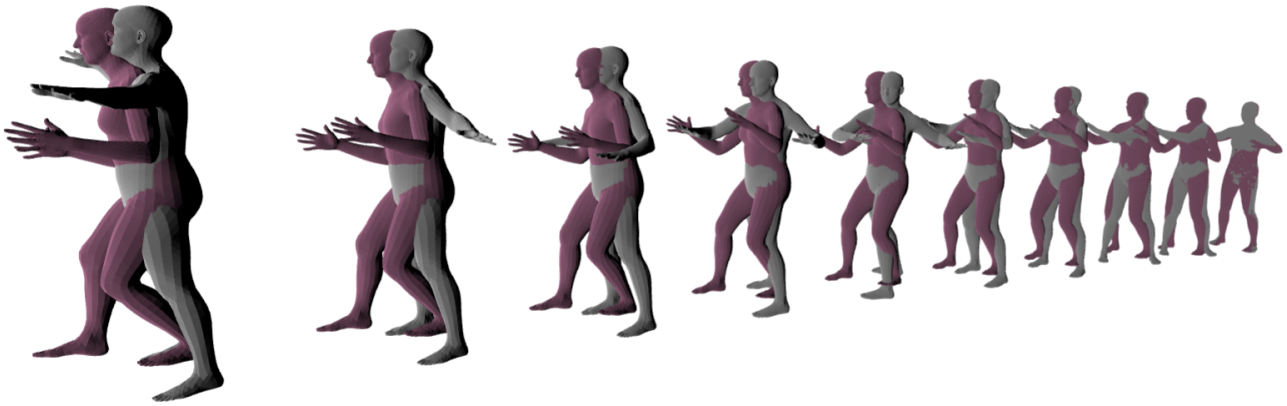

Figure 3: Average-case visualization at 5 seconds.

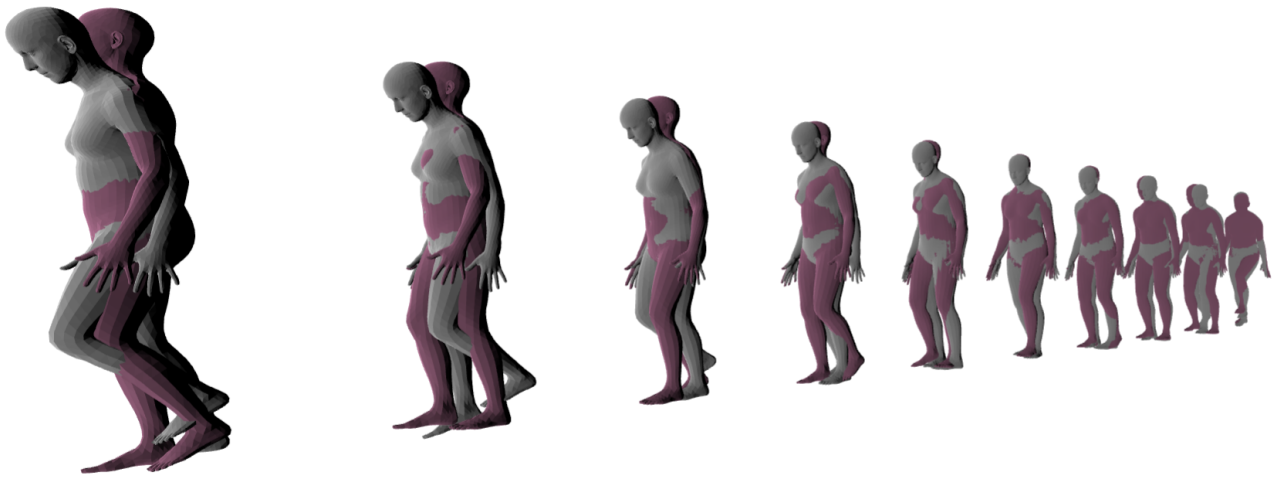

Figure 4: Best-case visualization at 5 seconds.